

第6章 图

目录

概述

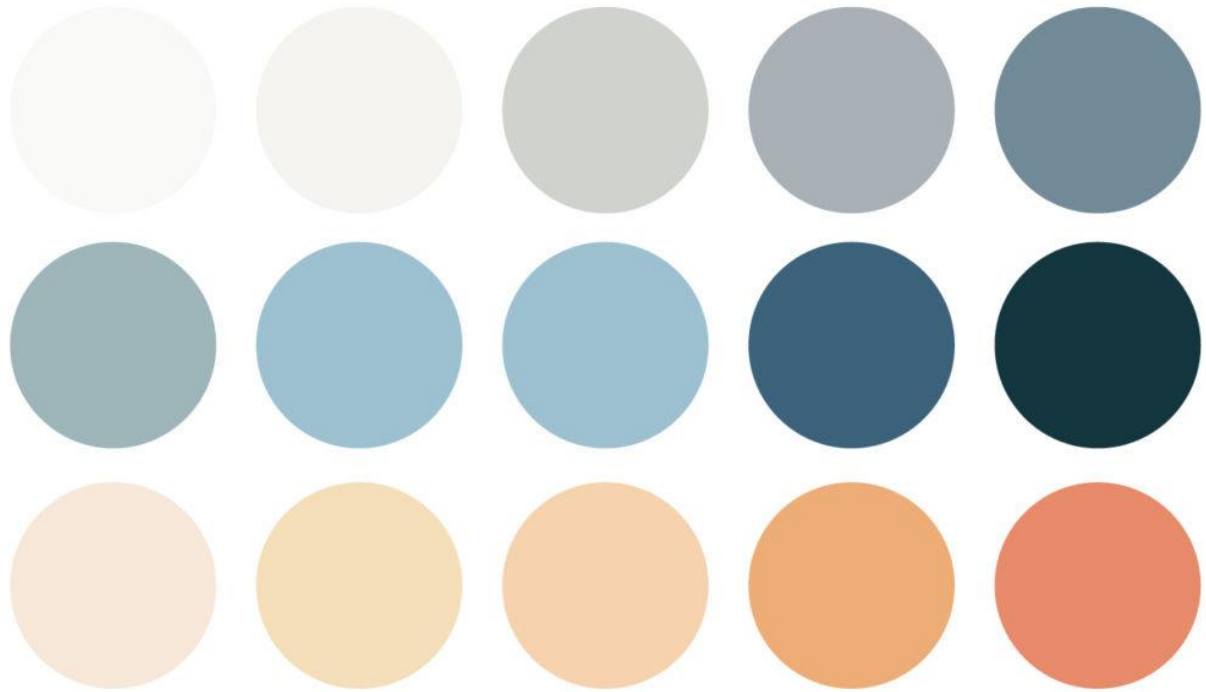
图的存储结构

图的遍历

最小生成树

最短路径

拓扑排序和关键路径



6.1 概述

- 在离散数学中，图论研究图的纯数学性质；
- 在数据结构中，图结构研究计算机中如何存储图以及如何实现图的操作和应用。
- 图是刻画离散结构的一种有力工具。在运筹规划、网络研究和计算机程序流程分析中都存在图的应用问题。我们也经常用图来表达文字难以描述的信息，如城市交通图、铁路网等。

6.1.1 图的基本概念

- 图是一种数据元素间具有“多对多”关系的非线性数据结构，由顶点集V和边集E组成，记作$G = (V, E)$。其中V是有穷非空集合，$v \in V$称为顶点； E是有穷集合，$e \in E$称为边。
- 与线性结构和树相比，图更为复杂。从数据间的逻辑关系来说，线性结构的数据元素间存在“一对一”的线性关系； 树的数据元素间存在层次关系，具有“一对多”的特性； 在图中每一个数据元素都可以和其他的任意数据元素相关。图中的每个元素可以有多个前驱元素和多个后继元素，任意两个元素可以相邻。

6.1.1 图的基本概念

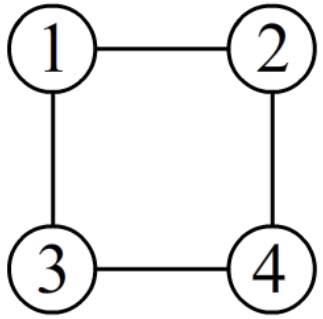
- **1. 无向边**
  - $e=(u,v)$表示顶点u和顶点v间的一条无向边，也可以简称为边。(u,v)间没有方向，即(u,v)和(v,u)是相同的。
- **2. 有向边**
  - $e=<u,v>$表示顶点u到顶点v间的一条有向边，也叫弧。u叫始点或弧尾，v叫终点或弧头。$<u,v>$是有方向的，因此$<u,v>$和$<v,u>$是不同的。
- **3. 零图**
  - 零图是指E为空集的图，也就是图中只有顶点存在，没有边。

6.1.1 图的基本概念

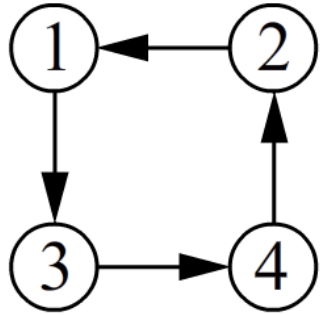
- **4. 无向图**
  - 无向图指全部由无向边构成的图。

- **5. 有向图**
  - 有向图指全部由有向边构成的图。

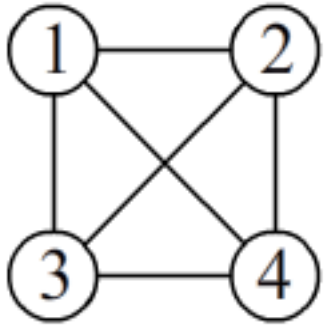
- **6. 完全图**
  - 完全图是指边数达到最大值的图，即在顶点数为n的无向图中边数为$n(n-1)/2$，在顶点数为n的有向图中边数为$n(n-1)$。



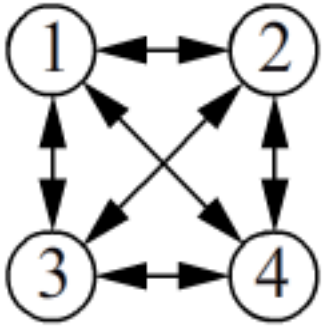
无向图



有向图



完全无向图



完全有向图

6.1.1 图的基本概念

- **7. 稠密图**
  - 稠密图是指边数较少的图，如$e < n\log_2 n$，反之则为稀疏图。
- **8. 子图**
  - 设有两个图$G = (V, E)$和$G' = (V', E')$，如果有$V' \subseteq V$和$E' \subseteq E$，则称G'是G的子图，记作$G' \subseteq G$。
- **9. 生成子图**
  - 如果$G' = (V', E')$是$G = (V, E)$的子图，并且$V' = V$，则称G'是G的生成子图。
- **10. 邻接点**
  - 在一个无向图中若存在边(u, v)，则称顶点u和v互为邻接点。边(u, v)是顶点u和v关联的边，顶点u和v是边(u, v)关联的顶点。

6.1.1 图的基本概念

- **11. 顶点的度**
  - 顶点的度是指与该顶点关联的边的数目。顶点u的度记作$D(u)$。
  - 在有向图中顶点的度有入度和出度两种。对于顶点u，入度指的是以u为终点的弧的数目，记为$ID(u)$； 出度指的是以u为起点的弧的数目，记为$OD(u)$。
  - 全部顶点的度之和为边数的两倍。
- **12. 路径**
  - 路径是指从顶点u到顶点v所经过的顶点序列。路径长度是指路径上边的数目。没有顶点重复出现的路径叫初等路径。
- **13. 回路**
  - 第一个和最后一个顶点相同的路径称为回路或环，除了第一个和最后一个顶点以外，其他顶点都不重复出现的回路叫初等回路。

6.1.1 图的基本概念

- **14. 连通图**
  - 在无向图中若顶点u和顶点v间有路径，则称u和v是连通的。连通图是指任意两个顶点均是连通的图。

- **15. 连通分量**
  - 连通分量是指无向图中的极大连通子图。

- **16. 强连通图**
  - 在有向图中若任意两个顶点均是连通的，则称该图为强连通图。

- **17. 强连通分量**
  - 强连通分量是指有向图中的极大连通子图。

6.1.1 图的基本概念

- **18. 生成树和生成森林**
  - 生成树是指包含图中的全部顶点，但只有构成树的$n-1$条边的生成子图。对于非连通图，每个连通分量可形成一棵生成树，所有生成树组成的集合叫该非连通图的生成森林。

- **19. 网**
  - 网指的是边上带有权值的图。通常权为非负实数，可以表示从一个顶点到另一个顶点的距离、时间和代价等。

6.1.2 图的抽象数据类型描述

- 图的抽象数据类型用Python抽象类描述如下：

```
from abc import ABCMeta, abstractmethod, abstractproperty

class IGraph(metaclass=ABCMeta):
    @abstractmethod
    def createGraph(self):
        '''创建图'''
        pass

    @abstractmethod
    def getVNum(self):
        '''返回图中的顶点数'''
        pass

    @abstractmethod
    def getENum(self):
        '''返回图中的边数'''
        pass
```

```
@abstractmethod
def getVex(self, i):
    '''返回位置为i的顶点值'''
    pass

@abstractmethod
def locateVex(self, x):
    '''返回值为x的顶点位置'''
    pass

@abstractmethod
def firstAdj(self, i):
    '''返回结点的第一个邻接点'''
    pass

@abstractmethod
def nextadj(self, i, j):
    '''返回相对于j的下一个邻接点'''
    pass
```

6. 2 图的存储结构

- 6.2.1 邻接矩阵
- 6.2.2 邻接表

6.2 概述

- 图的存储结构需要存储顶点的值以及与顶点相关联的顶点和边的信息。
- 顶点间没有次序关系，各条边之间也没有次序关系，但是表示和存储一个图必须约定好顶点次序。
- 边集合表达每对顶点间的邻接关系，是二维线性关系。矩阵的存储结构常见有邻接矩阵、邻接表、十字链表3种。
  - 1. 边采用顺序存储结构，用二维数组存储，称为图的邻接矩阵。
  - 2. 边采用链式存储结构，存储行的后继，即矩阵行的单链表，称为图的邻接表。
  - 3. 边采用链式存储结构，存储行和列的后继，即矩阵十字链表，称为图的邻接多重表。

6.2.1 邻接矩阵

- **图的邻接矩阵的存储结构**
  - 假设图$G=(V,E)$具有n个顶点，即$\{v_0,v_1,\dots,v_{n-1}\}$，那么图的邻接矩阵可定义如下：
  - $A[i][j]=\begin{cases}1, <v_i,v_j>\in E\text{或}(v_i,v_j)\in E\\ 0, <v_i,v_j>\notin E\text{且}(v_i,v_j)\notin E\end{cases}$，其中$0\le i,j<n$
  - 假设图$G=(V,E)$为网，且w_{ij}为边(v_i,v_j)或$<v_i,v_j>$上的权值，则网的邻接矩阵可定义如下：
  - $A[i][j]=\begin{cases}w_{ij}, <v_i,v_j>\in E\text{或}(v_i,v_j)\in E\\ \infty, <v_i,v_j>\notin E\text{且}(v_i,v_j)\notin E\end{cases}$，其中$0\le i,j<n$
  - 分析可得，在无向图的邻接矩阵中第i行或者第i列的非零元素的个数为第i个顶点的度；在有向图的邻接矩阵中第i行的非∞元素的个数为第i个顶点的出度，第i列非∞元素的个数为第i个顶点的入度。无向图的邻接矩阵是对称的，有向图的邻接矩阵不一定是对称的。

6.2.1 邻接矩阵

- 图的邻接矩阵可以用二维数组进行表示，邻接矩阵类的Python语言描述如下：

```
class MGraph(IGraph):
    # 图类别静态常量
    GRAPHKIND_UDG = 'UDG'
    GRAPHKIND_DG = 'DG'
    GRAPHKIND_UDN = 'UDN'
    GRAPHKIND_DN = 'DN'

    def __init__(self, kind=None, vNum=0, eNum=0, v=None, e=None):
        self.kind = kind # 图的种类
        self.vNum = vNum # 图的顶点数
        self.eNum = eNum # 图的边数
        self.v = v # 顶点列表
        self.e = e # 邻接矩阵
```

6.2.1 邻接矩阵

- 图的邻接矩阵类的基本操作的实现：1) 图的创建

```
# 算法6.1 创建无向图
    def createUDG(self, vNum, eNum, v, e):
        self.vNum = vNum
        self.eNum = eNum
        self.v = [None] * vNum # 构造顶点集
        for i in range(vNum):
            self.v[i] = v[i]
        self.e = [ [0] * vNum ] * vNum # 构造边集
        for i in range(eNum):
            a, b = e[i]
        m, n = self.locateVex(a), self.locateVex(b)
        self.e[m][n] = self.e[n][m] = 1
```

6.2.1 邻接矩阵

- 图的邻接矩阵类的基本操作的实现：1) 图的创建

```
# 算法6.2 创建有向图
    def createDG(self, vNum, eNum, v, e):
        self.vNum = vNum
        self.eNum = eNum
        self.v = [None] * vNum # 构造顶点集
        for i in range(vNum):
            self.v[i] = v[i]
        self.e = [ [0] * vNum ] * vNum # 构造边集
        for i in range(eNum):
            a, b = e[i]
        m, n = self.locateVex(a), self.locateVex(b)
        self.e[m][n] = 1
```

6.2.1 邻接矩阵

- 图的邻接矩阵类的基本操作的实现：1) 图的创建

```
# 算法6.3 创建无向网
    def createUDN(self, vNum, eNum, v, e):
        self.vNum = vNum
        self.eNum = eNum
        self.v = [None] * vNum # 构造顶点集
        for i in range(vNum):
            self.v[i] = v[i]
        self.e = [ [sys.maxsize] * vNum ] * vNum # 构造边集
        for i in range(eNum):
            a, b, w = e[i]
        m, n = self.locateVex(a), self.locateVex(b)
        self.e[m][n] = self.e[n][m] = w
```

6.2.1 邻接矩阵

- 图的邻接矩阵类的基本操作的实现：1) 图的创建

```
# 算法6.4 创建有向网
    def createDN(self, vNum, eNum, v, e):
        self.vNum = vNum
        self.eNum = eNum
        self.v = [None] * vNum # 构造顶点集
        for i in range(vNum):
            self.v[i] = v[i]
        self.e = [ [sys.maxsize] * vNum ] * vNum # 构造边集
        for i in range(eNum):
            a, b, w = e[i]
        m, n = self.locateVex(a), self.locateVex(b)
        self.e[m][n] = w
```

6.2.1 邻接矩阵

- **图的邻接矩阵类的基本操作的实现：2) 顶点的定位**
  - 顶点定位算法locateVex(x)是根据x的值取得其在顶点集中的位置，若不存在则返回-1。

```
# 算法6.5 顶点的定位
    def locateVex(self, x):
        for i in range(self.vNum):
            if self.v[i] == x:
                return i
        return -1
```

6.2.1 邻接矩阵

- **图的邻接矩阵类的基本操作的实现：3)查找第一个邻接点**
  - 查找第一个邻接点算法firstAdj(i)是指给定一个顶点在顶点集中的位置i，返回其第一个邻接点，若不存在则返回-1。

```
# 算法6.6 查找第一个邻接点
    def firstAdj(self, i):
        for i < 0 or i ≥ self.vNum:
            raise Exception("第%s个顶点不存在" % i)
        for j in range(self.vNum):
            if self.e[i][j] ≠ 0 and self.e[i][j] < sys.maxsize:
                return j
        return -1
```

6.2.1 邻接矩阵

- **图的邻接矩阵类的基本操作的实现：4) 查找下一个邻接点**
  - 查找下一个邻接点算法nextAdj(i,j)是指给定两个顶点在顶点集中的位置i、j，第j个顶点是第i个顶点的邻接点，返回第j个顶点之后的下一个邻接点，若不存在则返回-1。

```
# 算法6.6 查找第一个邻接点
    def firstAdj(self, i):
        for i < 0 or i ≥ self.vNum:
            raise Exception("第%s个顶点不存在" % i)
        for j in range(self.vNum):
            if self.e[i][j] ≠ 0 and self.e[i][j] < sys.maxsize:
                return j
        return -1
```

6.2.1 邻接矩阵

- **邻接矩阵表示图的性能分析**
  - 图的邻接矩阵表示存储了任意两个顶点间的邻接关系或边的权值，能够实现对图的各种操作，其中判断两个顶点间是否有边相连、获得和设置边的权值等操作的时间复杂度为$O(1)$。
  - 但是，与顺序表存储线性表的性能相似，由于采用数组存储，每插入或者删除一个元素需要移动大量元素，使得插入和删除操作的效率很低，而且数组容量有限，当扩充容量时需要复制全部元素，效率更低。
  - 在图的邻接矩阵中每个矩阵元素表示两个顶点间的邻接关系，无边或有边。即使两个顶点之间没有邻接关系，也占用一个存储单元存储0或者-1。对于一个有n个顶点的完全图，其邻接矩阵有$n(n-1)/2$个元素，此时邻接矩阵的存储效率较高；当图中的边数较少时，邻接矩阵变得稀疏，存储效率较低，此时可用图的邻接表进行存储。

6.2.1 邻接矩阵

- **【例6.1】n个顶点的无向图采用邻接矩阵存储，回答下列问题：**
  - （1） 图中有多少条边？
  - （2） 任意两个顶点i和j之间是否有边相连？
  - （3） 任意一个顶点的度是多少？
- **解：**
  - （1） 邻接矩阵中非零元素个数的总和除以2。
  - （2） 当邻接矩阵A中$A[i][j]=1$(或$A[j][i]=1$)时表示两顶点之间有边相连。
  - （3） 计算邻接矩阵上该顶点对应的行上非零元素的个数。

6.2.2 邻接表

- **图的邻接表的存储结构**
  - 邻接表采用链式存储结构存储图，是由一个顺序存储的顶点表和多个链式存储的边表组成的。
  - 边表的个数和图的顶点数相同。顶点表由顶点结点组成，每个顶点结点又由数据域和指针域组成，其中数据域data存放顶点值，指针域firstArc指向边表中的第一个边结点。
  - 边表由边结点组成，每个边结点又由adjVex、nextArc、value几个域组成，其中value存放边的信息，例如权值； adjVex存放与结点邻接的顶点在图中的位置；nextArc指向下一个边结点。

6.2.2 邻接表

- **图的邻接表的存储结构**
  - 邻接表的顶点结点类的Python描述如下：

```
class VNode(object):
    def __init__(self, data=None, firstNode=None):
        self.data = data # 存放节点值
        self.firstArc = firstNode # 第一条边
```

6.2.2 邻接表

- **图的邻接表的存储结构**
  - 邻接表的边结点类的Python描述如下：

```
class ArcNode(object):
    def __init__(self, adjVex, value, nextArc=None):
        self.adjVex = adjVex # 边指向的顶点的位置
        self.value = value # 边的权值
        self.nextArc =  nextArc # 指向下一条边
```

6.2.2 邻接表

- **图的邻接表的存储结构**
  - 图的邻接表类的描述如下：

```
class ALGraph(IGraph):
    # 图类别静态常
    GRAPHKIND_UDG = 'UDG'
    GRAPHKIND_DG = 'DG'
    GRAPHKIND_UDN = 'UDN'
    GRAPHKIND_DN = 'DN'

    def __init__(self, kind=None, vNum=0, eNum=0, v=None, e=None):
        self.kind = kind
        self.vNum = vNum
        self.eNum = eNum
        self.v = v
        self.e = e
```

```
def createGraph(self):
    if self.kind == self.GRAPHKIND_UDG:
        self.createUDG()
    elif self.kind == self.GRAPHKIND_DG:
        self.createDG()
    elif self.kind == self.GRAPHKIND_UDN:
        self.createUDN()
    elif self.kind == self.GRAPHKIND_DN:
        self.createDN()

def createUDG(self):
    '''创建无向
    pass'
def createDG(self):
    '''创建有向
    pass'

def createUDN(self):
    '''创建无向
    pass'

def createDN(self):
    '''创建有向
    pass'
```

6.2.2 邻接表

- **图的邻接表的存储结构**
  - 图的邻接表类的描述如下：

```
def addArc(self, i, j, value):
        '''插入边节点'''
        pass

    def firstAdj(self, i):
        '''查找第一个邻接点'''
        pass

    def nextAdj(self, i, j):
        '''返回i相对于j的下一个节点'''
        pass

    def getVNum(self):
        '''返回顶点数'''
        return self.vNum

    def getENum(self):
        '''返回边数'''
        return self.eNum
```

```
def getVex(self, i):
    '''返回第i个顶点的值'''
    if i < 0 or i ≥ self.vNum:
        raise Exception("第%s个顶点不存在" % i)
    return self.v[i].data

def locateVex(self, x):
    '''返回值为x的顶点的位置'''
    for i in range(self.vNum):
        if self.v[i].data == x:
            return i
    return -1

def getArcs(self, u, v):
    '''返回顶点u到顶点v的距离'''
    if u < 0 or u ≥ self.vNum:
        raise Exception("第%s个顶点不存在" % u)
    if v < 0 or v ≥ self.vNum:
        raise Exception("第%s个顶点不存在" % v)
    p = self.v[u].firstArc
    while p is not None:
        if p.adjVex == v:
            return p.value
        p = p.nextArc
    return sys.maxsize
```

6.2.2 邻接表

- 图的邻接表的基本操作的实现：1) 图的创建

```
# 算法6.8 创建无向图
    def createUDG(self):
        '''创建无向图'''
        v = self.v
        self.v = [ None ] * self.vNum
        for i in range(self.vNum):
            self.v[i] = VNode(v[i])
        for i in range(self.eNum):
            a, b = self.e[i]
            u, v = self.locateVex(a), self.locateVex(b)
            self.addArc(u, v, 1)
            self.addArc(v, u, 1)
```

6.2.2 邻接表

- 图的邻接表的基本操作的实现：1) 图的创建

```
# 算法6.9 创建有向图
    def createDG(self):
        '''创建有向图'''
        v = self.v
        self.v = [ None ] * self.vNum
        for i in range(self.vNum):
            self.v[i] = VNode(v[i])
        for i in range(self.eNum):
            a, b = self.e[i]
            u, v = self.locateVex(a), self.locateVex(b)
            self.addArc(u, v, 1)
```

6.2.2 邻接表

- 图的邻接表的基本操作的实现：1) 图的创建

```
# 算法6.10 创建无向网
    def createUDN(self):
        '''创建无向网'''
        v = self.v
        self.v = [ None ] * self.vNum
        for i in range(self.vNum):
            self.v[i] = VNode(v[i])
        for i in range(self.eNum):
            a, b = self.e[i]
            u, v = self.locateVex(a), self.locateVex(b)
            self.addArc(u, v, w)
            self.addArc(v, u, w)
```

6.2.2 邻接表

- 图的邻接表的基本操作的实现：1) 图的创建

```
# 算法6.11 创建有向网
    def createDN(self):
        '''创建有向网'''
        v = self.v
        self.v = [ None ] * self.vNum
        for i in range(self.vNum):
            self.v[i] = VNode(v[i])
        for i in range(self.eNum):
            a, b = self.e[i]
            u, v = self.locateVex(a), self.locateVex(b)
            self.addArc(u, v, w)
```

6.2.2 邻接表

- **图的邻接表的基本操作的实现：2)在图中插入边结点**
  - 插入边结点的算法addArc(i,j,value)是指在边链表中加入一个由第i个顶点指向第j个顶点的权值为value的边结点，采用头插法进行插入。

```
# 算法6.12 插入边结点
    def addArc(self, i, j, value):
        '''插入边节点'''
        arc = ArcNode(j, value)
        arc.nextArc = self.v[i].firstArc
        self.v[i].firstArc = arc
```

6.2.2 邻接表

- 图的邻接表的基本操作的实现：3)查找第一个邻接点

```
# 算法6.13 查找第一个邻接点
    def firstAdj(self, i):
        '''查找第一个邻接点'''
        if i < 0 or i ≥ self.vNum:
            raise Exception("第%s个顶点不存在" % i)
        p = self.v[i].firstArc
        if p is not None:
            return p.adjVex
        return -1
```

6.2.2 邻接表

- 图的邻接表的基本操作的实现：2)查找下一个邻接点

```
# 算法6.14 查找第一个邻接点
    def nextAdj(self, i, j):
        '''返回i相对于j的下一个节点'''
        if i < 0 or i ≥ self.vNum:
            raise Exception("第%s个顶点不存在" % i)
        p = self.v[i].firstArc
        while p is not None:
            if p.adjVex == j:
                break
            p = p.nextArc
        if p.nextArc is not None:
            return p.nextArc.adjVex
        return -1
```

6.2.2 邻接表

- **邻接矩阵表示图的性能分析**
  - 用邻接矩阵存储图可以很好地确定两个顶点间是否有边，但是查找顶点的邻接点，需要访问对应一行或一列的所有数据元素，并且无论两个顶点间是否有边都要保留存储空间。
  - 用邻接表存储图可以方便地找到顶点的邻接点，对于稀疏图来说节省存储空间，但若要确定两个顶点间是否有边相连则需要遍历单链表，比邻接矩阵复杂。

6. 3 图的遍历

- 6.1.1 图的基本概念
- 6.1.2 图的抽象数据类型描述

6.3 图的遍历

- 图的遍历是指从图的任意一个顶点出发对图的每个顶点访问且仅访问一次的过程，因为图中可能存在回路，为了避免对一个顶点的重复访问可以增设一个辅助数组visited[0 ... n-1]，全部初始化为0，一旦第i个顶点被访问，置visited[i]=1。图的遍历和树的遍历相比更加复杂，需要考虑以下3个问题。
  - (1) 指定遍历的第一个顶点。
  - (2) 由于一个顶点和多个顶点相邻，需要在多个邻接顶点间确定访问次序。
  - (3) 由于图中存在回路，必须对访问过的顶点做标记，防止出现重复访问同一顶点的情况。
- 图的遍历方式分为深度优先搜索和广度优先搜索两种。

6.3.1 图的广度优先搜索算法

- 广度优先搜索是一种分层的搜索过程，以一个顶点u为起始点访问其邻接点$v_0, v_1, \ldots$然后按顺序访问$v_0, v_1, \ldots$的各邻接点，重复此过程，即依次访问和顶点u之间存在路径并且路径长度为$1, 2, \ldots$的顶点。
- 图的广度优先搜索遵循“先被访问的顶点，其邻接点先被访问”规则，因此可引入队列。先将起始点加入队列中，以后每次从队列中删除一个数据元素，依次访问它的未被访问的邻接点，并将其插入到队列中，直到队列为空。

6.3.1 图的广度优先搜索算法

- 其主要步骤如下：
  - (1) 建立访问标识数组visited[n]并初始化为0,n为图顶点的个数。
  - (2) 将未访问顶点vi入队。
  - (3) 将队首元素顶点vi从队列中取出，依次访问它的未被访问的邻接点vj,vk,…，并将其入队。
  - (4) 重复步骤(3)，直到队列为空。
  - (5) 改变i值，0≤i<n，跳到步骤(2)继续进行，直到i=n-1。
- 假设图有n个顶点和m条边，当图的存储结构是邻接矩阵时需要扫描邻接矩阵的每一个顶点，其时间复杂度为$O(n^2)$； 当图的存储结构是邻接表时需要扫描每一条单链表，其时间复杂度为$O(e)$。

6.3.1 图的广度优先搜索算法

```
# 算法6.14 图的广度优先搜索
def BFSTraverse(g):
    global visited
    # 建立访问标志数组
    visited = [ False ] * g.getVNum()
    # 以每个顶点作为起始顶点进行遍历
    for i in range(g.getVNum()):
        if not visited[i]:
            BFS(g, i)

def BFS(g, i):
    # 建立辅助队列
    q = LinkQueue()
    # 将起点加入队列
    q.offer(i)
    while not q.isEmpty():
        u = q.poll()
        # 标记顶点已访问
        visited[u] = True
        print(g.getVex(u), end=' ')
        v = g.firstAdj(u)
        while v = -1:
            # 顶点未访问的邻接点入队
            if not visited[v]:
                q.offer(v)
            # 获取下一个邻接点
            v = g.nextAdj(u, v)
```

6.3.2 图的深度优先搜索算法

- 深度优先搜索类似于树的先序遍历，以一个顶点u为起始点访问其邻接点v_0，再访问v_0的未被访问的邻接点v_1，然后从v_1出发继续进行类似的访问，直到所有的邻接点都被访问。然后后退到前一个被访问的顶点，看是否有其他未被访问的顶点，若有再进行类似的访问，若无继续回退，直到图中的所有顶点都被访问为止。
- 其主要步骤如下。
  - (1) 建立访问标识数组`visited[n]`并初始化为0,n为图顶点的个数。
  - (2) 以未访问顶点`vi`为起始点访问其未访问邻接点`vj`。
  - (3) 从vj出发递归进行步骤(2)，直到所有邻接点均被访问。
  - (4) 改变i值，`0≤i<n`，跳到步骤(2)继续进行，直到`i=n-1`。

6.3.2 图的深度优先搜索算法

- 假设图有n个顶点和m条边，当图的存储结构是邻接矩阵时需要扫描邻接矩阵的每一个顶点，其时间复杂度为$O(n^2)$；当图的存储结构是邻接表时需要扫描每一条单链表，其时间复杂度为$O(e)$。

```
# 算法6.15 图的深度优先搜索
def DFSTraverse(g):
    global visited
    # 建立访问标志数组
    visited = [ False ] * g.getVNum()
    # 以每个顶点作为起始顶点进行遍历
    for i in range(g.getVNum()):
        if not visited[i]:
            DFS(g, i)

def DFS(g, i):
    visited[i] = True
    print(g.getVex(i), end=' ')
    v = v.firstAdj(i)
    while v ≠ -1:
        if not visited[v]:
            DFS(g, v)
        v = g.nextAdj(i, v)
```

6.3 图的遍历

- 【例6.2】编程利用广度优先搜索算法确定无向图的连通分量。
- 解：

```
def BFSTraverse(g):
    global visited
    # 建立访问标志数组
    visited = [ False ] * g.getVNum()
    count = 0
    # 以每个顶点作为起始顶点进行遍历
    for i in range(g.getVNum()):
        if not visited[i]:
            count += 1
            print("第%s个连通块：" % count, end=' ')
            BFS(g, i)
            print()

def BFS(g, i):
    # 建立辅助队列
    q = LinkQueue()
    # 将起点加入队列
    q.offer(i)
    while not q.isEmpty():
        u = q.poll()
        # 标记顶点已访问
        visited[u] = True
        print(g.getVex(u), end=' ')
        v = g.firstAdj(u)
        while v = -1:
            # 顶点未访问的邻接点入队
            if not visited[v]:
                q.offer(v)
            # 获取下一个邻接点
            v = g.nextAdj(u, v)

v = [1, 2, 3, 4, 5, 6]
e = [(1, 2), (2, 3), (4, 5)]
g = ALGraph(ALGraph, GRAPHKIND_UDG, len(v), len(e), v, e)
g.createGraph()
BFSTraverse(g)
```

6.3 图的遍历

- 【例6.2】编程利用广度优先搜索算法确定无向图的连通分量。
- 输出如下
  - 第1个连通块： 1 2 3
  - 第2个连通块： 4 5
  - 第3个连通块： 6

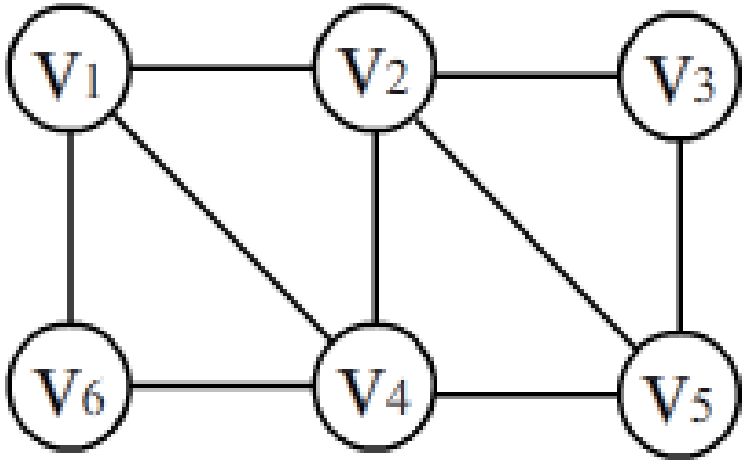
```
def BFSTraverse(g):
    global visited
    # 建立访问标志数组
    visited = [ False ] * g.getVNum()
    count = 0
    # 以每个顶点作为起始顶点进行遍历
    for i in range(g.getVNum()):
        if not visited[i]:
            count += 1
            print("第%s个连通块: " % count, end=' ')
            BFS(g, i)
            print()

def BFS(g, i):
    # 建立辅助队列
    q = LinkQueue()
    # 将起点加入队列
    q.offer(i)
    while not q.isEmpty():
        u = q.poll()
        # 标记顶点已访问
        visited[u] = True
        print(g.getVex(u), end=' ')
        v = g.firstAdj(u)
        while v = -1:
            # 顶点未访问的邻接点入队
            if not visited[v]:
                q.offer(v)
            # 获取下一个邻接点
            v = g.nextAdj(u, v)

v = [1, 2, 3, 4, 5, 6]
e = [(1, 2), (2, 3), (4, 5)]
g = ALGraph(ALGraph, GRAPHKIND_UDG, len(v), len(e), v, e)
g.createGraph()
BFSTraverse(g)
```

6.3 图的遍历

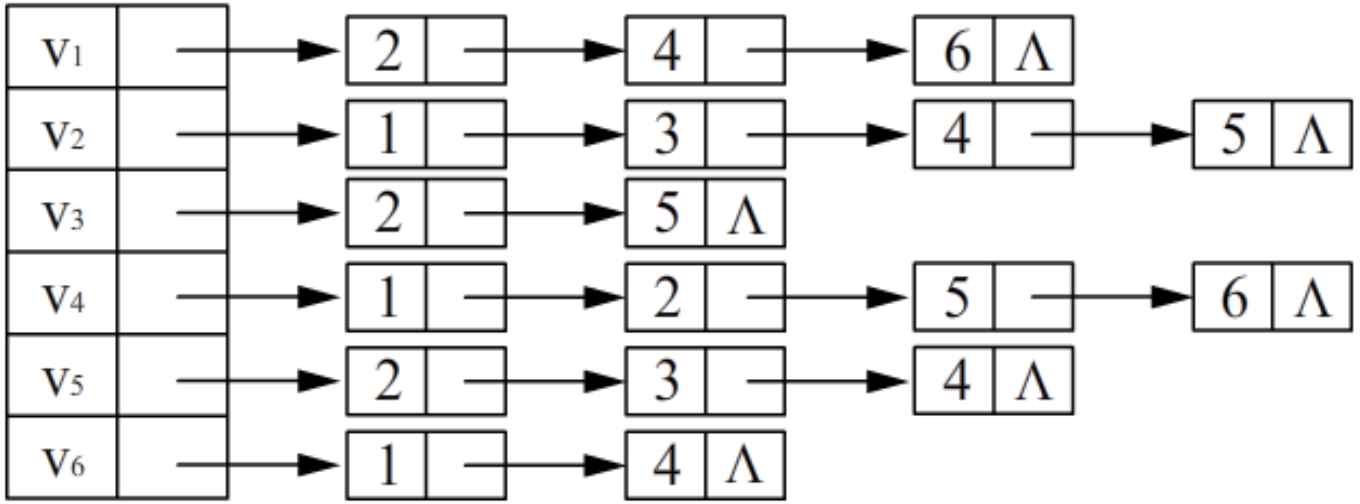
- **【例6.3】已知一个连通图如下所示，试给出图的邻接矩阵和邻接表存储示意图，若从顶点v1出发对该图进行遍历，分别给出一个按深度优先遍历和广度优先遍历的顶点序列。**



连通图

6.3 图的遍历

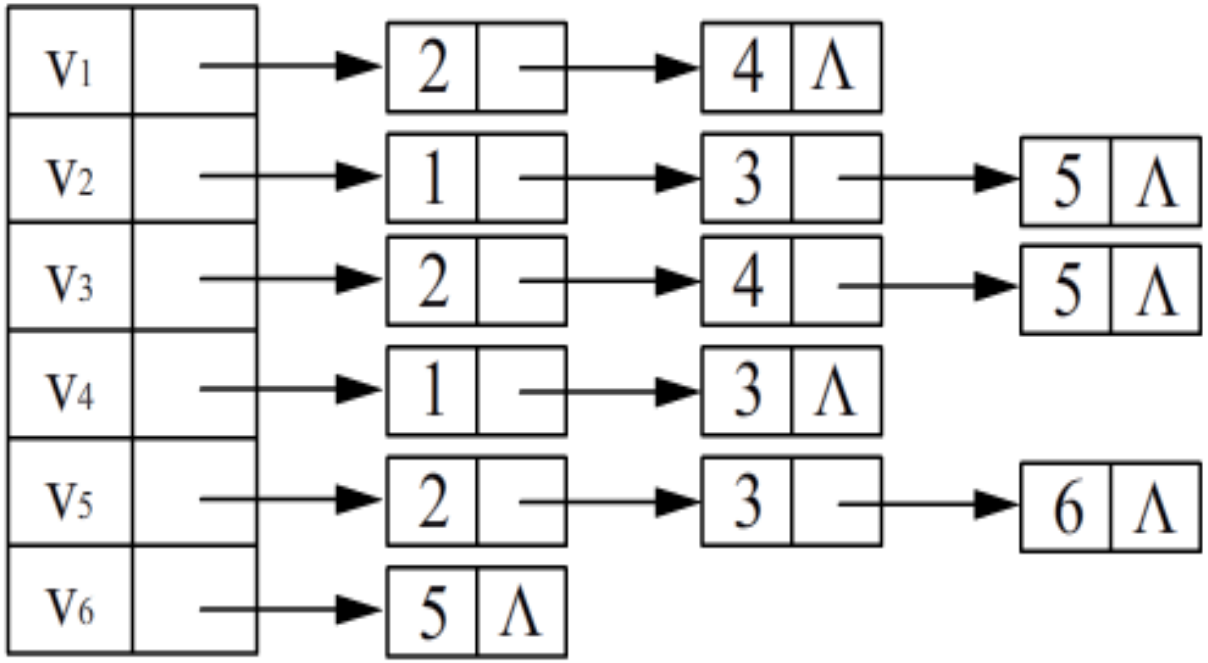
- 【例6.3】已知一个连通图如下所示，试给出图的邻接矩阵和邻接表存储示意图，若从顶点v1出发对该图进行遍历，分别给出一个按深度优先遍历和广度优先遍历的顶点序列。
- 解：
  - $$\begin{matrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{matrix}$$



- 深度优先遍历序列：$v_1\ v_2\ v_3\ v_4\ v_5\ v_6$
- 广度优先遍历序列：$v_1\ v_2\ v_4\ v_6\ v_3\ v_5$

6.3 图的遍历

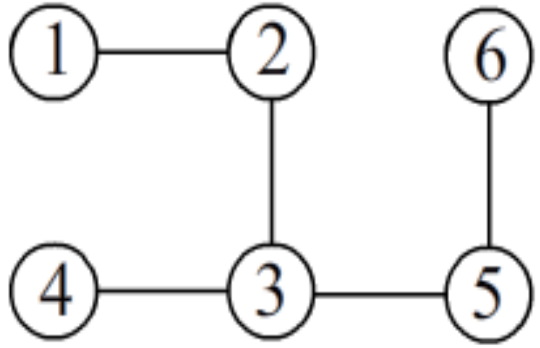
- 【例6.4】已知无向图G的邻接表如图所示，分别写出从顶点1出发的深度遍历和广度遍历序列，并画出相应的生成树。



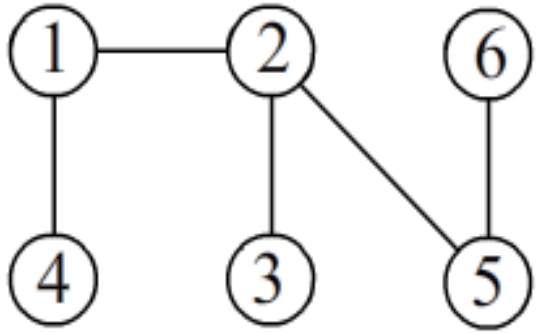
无向图G的邻接表

6.3 图的遍历

- 【例6.4】已知无向图G的邻接表如图所示，分别写出从顶点1出发的深度遍历和广度遍历序列，并画出相应的生成树。
- 解：
  - 由章节给出的算法可知深度优先遍历序列为$1, 2, 3, 4, 5, 6$。
  - 对应的生成树如图所示。
  - 广度优先遍历序列为$1, 2, 4, 3, 5, 6$。
  - 对应的生成树如图所示。



深度优先遍历序列对应的生成树



广度优先遍历序列对应的生成树

6.4 最小生成树

- 6.4.1 最小生成树的基本概念
- 6.4.2 Kruskal算法
- 6.4.3 Prim算法

6.4.1 最小生成树的基本概念

- 连通图的生成树是图的极小连通子图，它包含图中的全部顶点，但只有构成一棵树的边。一个有n个顶点的连通图的生成树只有n-1条边。若有n个顶点而少于n-1条边，则为非连通图，若多于n-1条边，则一定形成回路。
- 由广度优先遍历和深度优先遍历得到的生成树分别称为广度优先生成树和深度优先生成树。根据遍历方法的不同或遍历起点的不同得到的生成树也是不同的，图的生成树不唯一。
- 对于非连通图，每个连通分量中的顶点集和遍历经过的边一起构成若干棵生成树，共同组成了该非连通图的生成森林。

6.4.1 最小生成树的基本概念

- 在一个网的所有生成树中权值总和最小的生成树称为最小代价生成树，简称为最小生成树，最小生成树不一定唯一，需要满足以下3条准则。
  - (1) 只能使用图中的边构造最小生成树。
  - (2) 具有n个顶点和n-1条边。
  - (3) 不能使用产生回路的边。
- 产生最小生成树的方法主要有Kruskal算法和Prim算法两种。

6.4.2 Kruskal算法

- Kruskal算法是依次找出权值最小的边建立最小生成树，每次新增的边不能使生成树产生回路，直到找到n-1条边。

- 设图是由n个顶点组成的连通无向网，是图G的最小生成树，其中V是T的顶点集，TE是T的边集。构造最小生成树的步骤如下。

  ① 将T的初始状态置为仅含有源点的集合。

  ② 在图G的边集中选取权值最小的边，若该边未使生成树T形成回路，则加入到TE中，否则丢弃，直到生成树中包含了n-1条边。

- Kruskal算法的执行时间主要取决于图的边数，时间复杂度为$O(n^2)$，因此该算法适用于稀疏图的操作。

6.4.3 Prim算法

- 在介绍Prim算法之前需要先了解距离的概念。

① 两个顶点之间的距离： 将顶点u邻接到顶点v的关联边的权值，记为 $|u,v|$ 。若两个顶点之间不相连，则这两个顶点之间的距离为无穷大。

② 顶点到顶点集合的距离： 顶点到顶点集合中所有顶点的距离的最小值，记为$|u,V| = \min|u,v|$。

③ 两个顶点集合之间的距离： 顶点集合U的顶点到顶点集合V的距离的最小值，记为$|U,V| = \min|u,V|$。

6.4.3 Prim算法

- 设图是由n个顶点组成的连通无向网，是图G的最小生成树，其中V是T的顶点集，TE是T的边集，构造最小生成树的步骤为从开始，必存在一条边，使得U、V之间的距离最小,将加入到集合TE中，同时将顶点加入到顶点集U中，直到U=V为止。

- 针对每一个顶点引入分量closedge[i],它包含两个域，lowcost域存储该边上的权值，即顶点到顶点集U的距离； adjvex域存储该边在顶点集U中的顶点。因为集合U是随着数据元素的加入而逐渐增大的，所以有新的数据元素加入时将closedge[i].lowcost与到新的数据元素的距离进行比较即可。

6.4.3 Prim算法

Prim算法构造最小生成树的类描述如下：

```
class CloseEdge(object):
    def __init__(self,adjVex,lowCost):
        self.adjVex = adjVex    # 在集合U中的顶点的值
        self.lowCost = lowCost  # 到集合U的最小距离

class MiniSpanTree(object):

    def PRIM(g,u):
        '''从值为u的顶点出发构造最小生成树，返回由生成树边组成的二维数组'''
        # 存放生成树边上的顶点
        tree = [ [None,None] for i in range(g.getVNum() - 1) ]
        count = 0
        closeEdge = [ None ] * g.getVNum()
        k = g.locateVex(u)
        for j in range(g.getVNum()):
            if k != j:
                closeEdge[j] = CloseEdge(u,g.getArcs(k,j))
        # 将u添加到U中
        closeEdge[k] = CloseEdge(u,0)
        for i in range(1,g.getVNum()):
            # 找出具有到集合U最小距离的顶点的序号
            k = MiniSpanTree.getMinMum(closeEdge)
            tree[count][0] = closeEdge[k].adjVex # 在集合U中的顶点
            tree[count][1] = g.getVex(k) # 在集合V-U中的顶点
            count += 1
            # 更新 closeEdge
            closeEdge[k].lowCost = 0
            for j in range(g.getVNum()):
                if g.getArcs(k,j) < closeEdge[j].lowCost:
                    closeEdge[j] = CloseEdge(g.getVex(k),g.getArcs(k,j))
        return tree
    def getMinMum(closeEdge):
        '''选出lowCost最小的顶点'''
        minValue = sys.maxsize
        v = -1
        for i in range(len(closeEdge)):
            if closeEdge[i].lowCost != 0 and closeEdge[i].lowCost < minValue:
                minValue = closeEdge[i].lowCost
                v = i
        return v
```

6.5 最短路径

- 6.5.1 单源最短路径
- 6.5.2 求任意两个顶点间的最短路径

6.5 最短路径

- 最短路径的求解问题主要分为两类，即求某个顶点到其余顶点的最短路径，以及求每一对顶点间的最短路径，本节针对这两类问题提出了两种算法。

① 单源最短路径： Dijkstra算法

② 任意两顶点间最短路径：Floyd算法

6.5.1 单源最短路径

- 针对这一最短路径问题本节提出了Dijkstra算法，基本思想是“按最短路径长度递增的次序”产生最短路径。
- 若从源点到某个终点存在路径，则一定存在最短路径。从源点到其余各顶点的最短路径长度不一定一样，具有以下特点。

① 在这些最短路径中长度最短的最短路径一定有且仅有一条弧，弧的权值是从源点出发的所有弧的权中的最小值。

② 长度次短的最短路径有两种情况： 其一。只包含一条从源点出发的弧，弧上的权值大于已求得最短路径的弧的权值，小于其他从源点出发的弧的权值；其二，一条只经过已求得最短路径的顶点的路径。

6.5.1 单源最短路径

- 算法的主要步骤为保存当前已经得到的从源点到各个其余顶点的最短路径，也就是说，若源点到该顶点有弧，存在一条路径，长度为弧上的权值，每求得一条到达某个顶点的最短路径就需要检查是否存在经过这个顶点的其他路径，若存在，判断其长度是否比当前求得的路径长度短，若是，则修改当前路径。在算法中引入一个辅助向量D，它的每个分量D[i]存放当前所找到的从源点到终点的最短路径长度。

6.5.1 单源最短路径

Dijkstra算法构造最短路径的类Python语言描述如下：

```
class ShortestPath(object):
    def Dijkstra(g,v0):
        # 存放最短路径，p[v][k]表示从v0到v的最短路径中经过的第k个点
        p = [ ([ -1 ] * g.getVNum()) for i in range(g.getVNum()) ]
        # 存放最短路径长度
        D = [ sys.maxsize ] * g.getVNum()
        # 若已找到最短路径，finish[v]为True
        finish = [ False ] * g.getVNum()
        v0 = g.locateVex(v0)
        for v in range(g.getVNum()) :
            D[v] = g.getArcs(v0,v)
            if D[v] < sys.maxsize:
                # 从起点直接可以到达
                p[v][0] = v0
                p[v][1] = v0
        p[v0][0] = v0 # 起点本身可以直接到达
        D[v0] = 0
        finish[v0] = True
        v = -1
```

```
for i in range(1,g.getVNum()):
            minValue = sys.maxsize
            for w in range(g.getVNum()):
                # 找出所有最短路径中的最小值
                if not finish[w]:
                    if D[w] < minValue:
                        v = w
                        minValue = D[w]
            finish[v] = True
            # 更新当前的最短路径
            for w in range(g.getVNum()):
                if not finish[w] and g.getArcs(v,w) < sys.maxsize and
 (minValue + g.getArcs(v,w) < D[w]):
                    D[w] = minValue + g.getArcs(v,w)
                    for k in range(g.getVNum()):
                        p[w][k] = p[v][k]
                        if p[w][k] == -1:
                            p[w][k] = w
                            break
        dis = {g.getVex(i):D[i] for i in range(g.getVNum())}
        # 返回到各点最短路的字典与路径矩阵
        return dis,p
```

6.5.1 单源最短路径

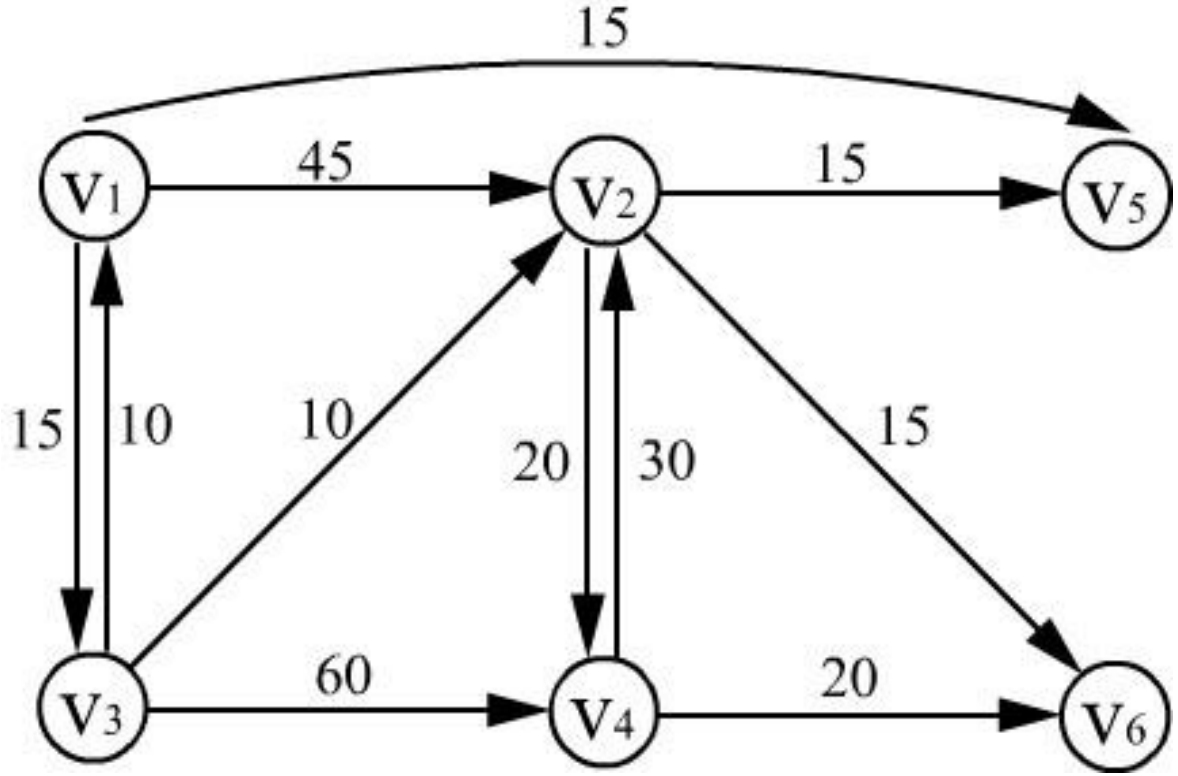
Dijkstra算法构造最短路径的类Python语言描述如下：

```
def printDijkstraPath(g,v0,p):
        # v0到各点的输出最短路径，即p[v][0]到p[v][j]直到p[v][j] == -1
        u = v0
        v0 = g.locateVex(v0)
        for i in range(g.getVNum()):
            v = g.getVex(i)
            print("%s->%s的最短路径为:" % (u,v),end=' ')
            if p[i][0] != -1:
                print(g.getVex(p[v0][0])),end='')
            for k in range(1,g.getVNum()):
                if p[i][k] == -1:
                    break
                print("->%s" % g.getVex(p[i][k]),end='')
            print()
```

分析可得，Dijkstra算法的时间复杂度为$O(n^2)$，并且找到一条从源点到某一特定终点之间的最短路径，和求从源点到各个终点的最短路径一样复杂，时间复杂度也为$O(n^2)$。

6.5.1 单源最短路径

- 【例6.5】假设有如图所示的有向网图，利用Dijkstra算法求从顶点v1到其他各顶点的最短路径。



有向网图

6.3 图的遍历

- **【例6.5】假设有如图所示的有向网图，利用Dijkstra算法求从顶点v1到其他各顶点的最短路径。**
- 解：从源点v1到其他各顶点的最短路径如表所示。

源点	终点	最短路径	最短路径长度
v_1	v_3	$v_1\ v_3$	15
v_1	v_5	$v_1\ v_5$	15
v_1	v_2	$v_1\ v_3\ v_2$	25
v_1	v_6	$v_1\ v_3\ v_2\ v_6$	40
v_1	v_4	$v_1\ v_3\ v_2\ v_4$	45

6.5.2 求任意两个顶点间的最短路径

- 求任意两个顶点间的最短路径，如果使用Dijkstra算法，可以依次将顶点设为源点，调用算法n次即可求得，时间复杂度为$O(n^3)$。本节讲解了算法形式更为简单的Floyd算法，时间复杂度也为$O(n^3)$。用户可以用n阶方阵序列来描述Floyd算法，其中$D^{-1}[i][j]$表示从顶点出发不经过其他顶点直接到达顶点的路径长度，即$D^{-1}[i][j]=G.arcs[i][j]$，$D^{(k)}[i][j]$表示从顶点$v_i$到顶点$v_j$的中间可能经过$v_0,\dots,v_k$而不可能经过$v_{k+1},\dots,v_{n-1}$等顶点的最短路径长度，所以$D^{(n-1)}[i][j]$是从顶点$v_i$到顶点$v_j$的最短路径长度，和路径长度序列相对应的是路径的n阶方阵序列$p^{(-1)},p^{(0)},p^{(1)},\dots,p^{(n-1)}$。

6.5.2 求任意两个顶点间的最短路径

所以，Floyd算法的基本操作可以概括为：

```
if D[i][k] + D[k][j] < D[i][j]:
    D[i][j] = D[i][k] + D[k][j]
    P[i][j] = P[i][k] + P[k][j]
```

其中，k表示在路径中新增的顶点，i为路径的源点，j为路径的终点。

6.5.2 求任意两个顶点间的最短路径

Floyd算法构造最短路径的类用Python语言描述如下：

```
def Floyd(g):
    vNum=g.getVNum()
    D = [ [ sys.maxsize ] * vNum for i in range(vNum) ]
    p = [ [ -1 ] * vNum for i in range(vNum) ]
    for u in range(vNum):
        for v in range(vNum):
            D[u][v] = g.getArcs(u,v) if u != v else 0
            if D[u][v] < sys.maxsize:
                p[u][v] =  u
    for k in range(vNum):
        for i in range(vNum):
            for j in range(vNum):
                if D[i][j] > D[i][k] + D[k][j]:
                    D[i][j] = D[i][k] + D[k][j]
                    p[i][k]=i
                    p[i][j] = p[k][j]
    dis = {(g.getVex(u),g.getVex(v)):D[u][v] for u in range(vNum) for v in range(vNum)}
    return dis,p
```

6.5.2 求任意两个顶点间的最短路径

Floyd算法构造最短路径的类用Python语言描述如下：

```
def printFloydPath(g,p):
    vNum=g.getVNum()
    for u in range(vNum):
        for v in range(vNum):
            if u == v:
                print("%s->%s的最短路径为:%s" % (g.getVex(u),g.getVex(v),g.getVex(u)))
                continue
            flag=True
            path=[v]
            t = p[u][path[0]]
            while t != u:
                if t == -1:
                    flag = False
                    break
                path = [t] + path
                t = p[u][t]
            print("%s->%s的最短路径为:" % (g.getVex(u),g.getVex(v)),end='')
            if flag:
                print(g.getVex(u),end='')
                for node in path:
                    print('->%s' % g.getVex(node),end='')
            print()
```

6.6 拓扑排序和关键路径

- 6.6.1 拓扑排序
- 6.6.2 关键路径

6.6 拓扑排序和关键路径

- 在生产实践中，几乎所有的工程都可以分解为若干个具有相对独立性的子工程，称为“活动”。活动之间又通常受到一定条件的约束，即某些活动必须在另一些活动完成之后才能进行。可以使用有向图表示活动之间相互制约的关系，顶点表示活动，弧表示活动之间的优先关系，这种有向图称为顶点活动网(AOV)。若在AOV网中存在一条从顶点u到顶点v的弧，则活动u一定优先于活动v发生，否则活动u、v的发生顺序可以是任意的。

- 在AOV网中不允许出现环，否则某项活动的进行以其本身的完成作为先决条件，这是不允许发生的。判断有向网中是否存在环的方法是进行拓扑排序。

6.6.1 拓扑排序

- 对AOV网进行拓扑排序即构造一个包含图中所有顶点的拓扑有序序列，若在AOV网中存在一条从顶点u到顶点v的弧，则在拓扑有序序列中顶点u必须先于顶点v，否则顶点u、v的顺序可以是任意的。AOV网的拓扑有序序列并不唯一。若AOV网中存在环，则不可能将所有的顶点都纳入到拓扑有序序列中，因此可以用拓扑排序判断有向网中是否存在环。拓扑排序的主要步骤如下。

  (1) 在AOV网中选择一个没有前驱的顶点并输出。

  (2) 从AOV网中删除该顶点以及从它出发的弧。

  (3) 重复步骤(1)和(2)直到AOV网为空，或者剩余子图中不存在没有前驱的顶点，此时说明AOV网中存在环。

- 整个拓扑排序可以分成求各个顶点的入度和一个拓扑序列的过程，具体算法描述如下。

6.6.1 拓扑排序

```
# 算法6.16 求各顶点的入度
def findInDegree(g):
    indegree = [ 0 ] * g.getVNum()
    # 计算每个点的入度
    for u in range(g.getVNum()):
        v = g.firstAdj(u)
        while v != 1:
            indegree[v] += 1
            v = g.nextAdj(u,v)
    return indegree
```

6.6.1 拓扑排序

```
# 6.17 计算AOV的一个拓扑序列，
# 若存在返回拓扑序列，否则返回None
def topoSort(g):
    count = 0
    indegree = findInDegree(g)
    s = LinkStack()
    topo = []
    for i in range(g.getVNum()):
        # 入度为零的点入栈
        if indegree[i] == 0:
            s.push(i)
        while not s.isEmpty():
            u = s.pop()
            topo.append(u)
            count += 1
            # 对该点的每个邻接点的入度-1
            v = g.firstAdj(u)
            while v != -1:
                indegree[v] -= 1
                if indegree[v] == 0:
                    s.push(v)
                v = g.nextAdj(u,v)
    if count < g.getVNum():
        return None
    return [g.getVex(u) for u in topo]
```

6.6.2 关键路径

- 若以弧表示活动，弧上的权值表示进行该项活动需要的时间，顶点表示事件，这种有向网称为边活动网络，简称为AOE网。弧指向事件表示该弧代表的活动已经完成，弧从事件出发表示该弧代表的活动开始进行，所以AOE网不允许环的存在。
- AOE网常用来表示工程的进行，表示工程开始事件的顶点的入度为0，称为源点，表示工程结束事件的顶点的出度为0，称为汇点，一个工程的AOE网应该是只有一个源点和一个汇点的有向无环图。由于AOE网中的某些活动可以并行进行，故完成整个工程的最短时间即从源点到汇点的最长路径的长度，这条路径称为关键路径，构成关键路径的弧即为关键活动。
- 假设V0为源点，Vn-1为汇点，事件的发生时刻为0时刻。从V0到Vi的最长路径叫做事件的最早发生时间，e(i)表示活动的最早开始时间，l(i)表示活动的最晚开始时间，指的是在不推迟整个工程的前提下，活动最晚必须开始的时间。当e(i)=l(i),称为关键活动。提前完成非关键活动并不能加快工程的进度，如果要缩短整个工期，必须首先找到关键路径，提高关键活动的工效。

6.6.2 关键路径

- 根据事件的最早发生时间和最晚发生时间的定义可以采用下列步骤求得关键活动。
  - (1)从源点出发，令ve(0)=0，其余各顶点的ve(j)=max(ve(i)+|i,l|)，其中，T是所有以第j个顶点为头的弧的集合。若得到的拓扑排序序列中顶点的个数小于网中的顶点个数n，则说明网中有环，不能求出关键路径，算法结束。
  - (2) 从Vn-1汇点出发，令vl(n-1)=ve(n-1)，按逆拓扑排序求其余各顶点允许的最晚开始时间为vl(i)=min(vl(j)-|i,j|)，其中，S是所有以第j个顶点为尾的弧的集合。
  - (3)每一项活动ai的最早开始时间为e(i)=ve(j)，最晚开始时间为l(i)=vl(j)-|i,j|。若ai满足e(i)=l(i),则它是关键活动。

6.6.2 关键路径

```
# 算法6.18 若拓扑序列存在，返回各顶点的最早发生
# 时间ve与逆拓扑序列栈t，否则返回None,None
def topoOrder(g):
    count = 0
    indegree = findInDegree(g)
    s = LinkStack()
    t = LinkStack()
    ve = [ 0 ] * g.getVNum()
    for i in range(g.getVNum()):
        # 入度为零的点入栈
        if indegree[i] == 0:
            s.push(i)
    while not s.isEmpty():
        u = s.pop()
        t.push(u)
        coount += 1
        # 对该点的每个邻接点的入度-1
        v = g.firstAdj(u)
        while v != -1:
            indegree[v] -= 1
            if indegree[v] == 0:
                s.push(v)
            # 更新最早发生时间
            if ve[u] + g.getArcs(u,v) > ve[v]:
                ve[v] = ve[u] + g.getArcs(u,v)
            v = g.nextAdj(u,v)
    if count < g.getVNum():
        return None,None
    return ve,t
```

6.6.2 关键路径

```
# 算法6.19 求各顶点的最晚发生时间并输出关键活动
def criticalPath(g):
    ve,t = topoOrder(g)
    if ve is None:
        return None
    vl = [ ve[t.top.data] ] * g.getVNum()
    # 逆拓扑序求各个顶点的vl值
    while not t.isEmpty():
        u = t.pop()
        v = g.firstAdj(u)
        while v != -1:
            if vl[v] - g.getArcs(u,v) < vl[u]:
                vl[u] = vl[v] - g.getArcs(u,v)
            v = g.nextAdj(u,v)
    # 输出关键活动
    print("关键活动为: ",end='')
    for i in range(g.getVNum()):
        if ve[i] == vl[i]:
            print(g.getVex(i),end=' ')
    print()
```

小结

小结

- (1) 图是一种数据元素间具有“多对多”关系的非线性数据结构，由顶点集V和边集E组成，记作G=(V,E)。
- (2) 图的常见存储结构有邻接矩阵、邻接表、十字链表3种。邻接矩阵是图，用二维数组存储； 邻接表和十字链表是图的链式存储结构。
- (3) 图的遍历是指从图的任意一个顶点出发对图的每个顶点访问且仅访问一次的过程。图的遍历方式分为两种，即广度优先搜索遍历和深度优先搜索遍历。
- (4) 由广度优先遍历和深度优先遍历得到的生成树分别称为广度优先生成树和深度优先生成树。在一个网的所有生成树中权值总和最小的生成树称为最小代价生成树，简称为最小生成树，最小生成树不一定唯一。建立最小生成树的方法有Kruskal算法和Prim算法。

小结

- (5) 最短路径的求解问题主要分为两类，即求某个顶点到其余顶点的最短路径、求每一对顶点间的最短路径，可以分别使用Dijkstra算法和Floyd算法解决这两类问题。
- (6) 用户可以使用有向图表示活动之间相互制约的关系，顶点表示活动，弧表示活动之间的优先关系，这种有向图称为顶点活动网(AOV)。若在AOV网中存在一条从顶点u到顶点v的弧，则活动u一定优先于活动v发生。
- (7) 若以弧表示活动，弧上的权值表示进行该项活动需要的时间，顶点表示事件，这种有向网称为边活动网络，简称为AOE网。AOE网络常用来表示工程的进行，一个工程的AOE网应该是只有一个源点和一个汇点的有向无环图。由于AOE网中的某些活动可以并行进行，故完成整个工程的最短时间即从源点到汇点的最长路径的长度，这条路径称为关键路径，构成关键路径的弧即为关键活动。